CONTENT

WHAT ARE REACT NATIVE COMPONENTS ?

STYLING COMPONENTS USING STYLESHEET

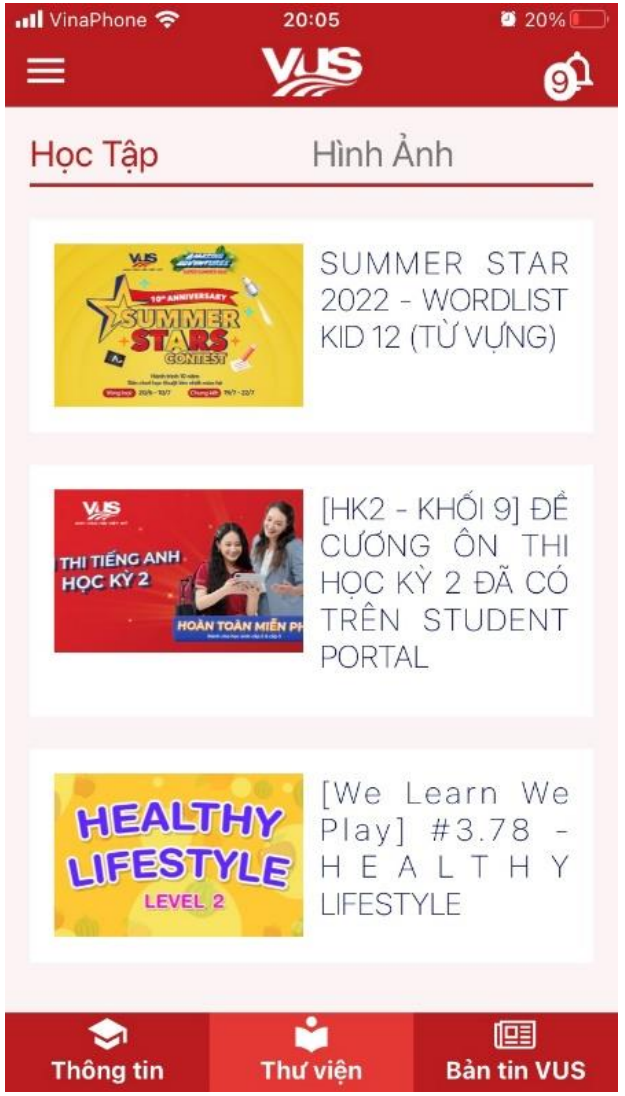
REACT NATIVE HOOK

BUIDING CUSTOM COMPONENT

1

WHAT ARE REACT NATIVE COMPONENTS?

A basic building block, reusable to build the UI



Header→

CÙNG #VUS SUPERKIDS FESTIVAL!
21/06/2022

VUS SuperKids Festival sẽ chính thức **MỞ CỔNG MIỄN PHÍ từ 22/6/2022** tại sảnh Tây **AEON MALL Tân Phú Celadon!**

Footer→

Menu→

Item→

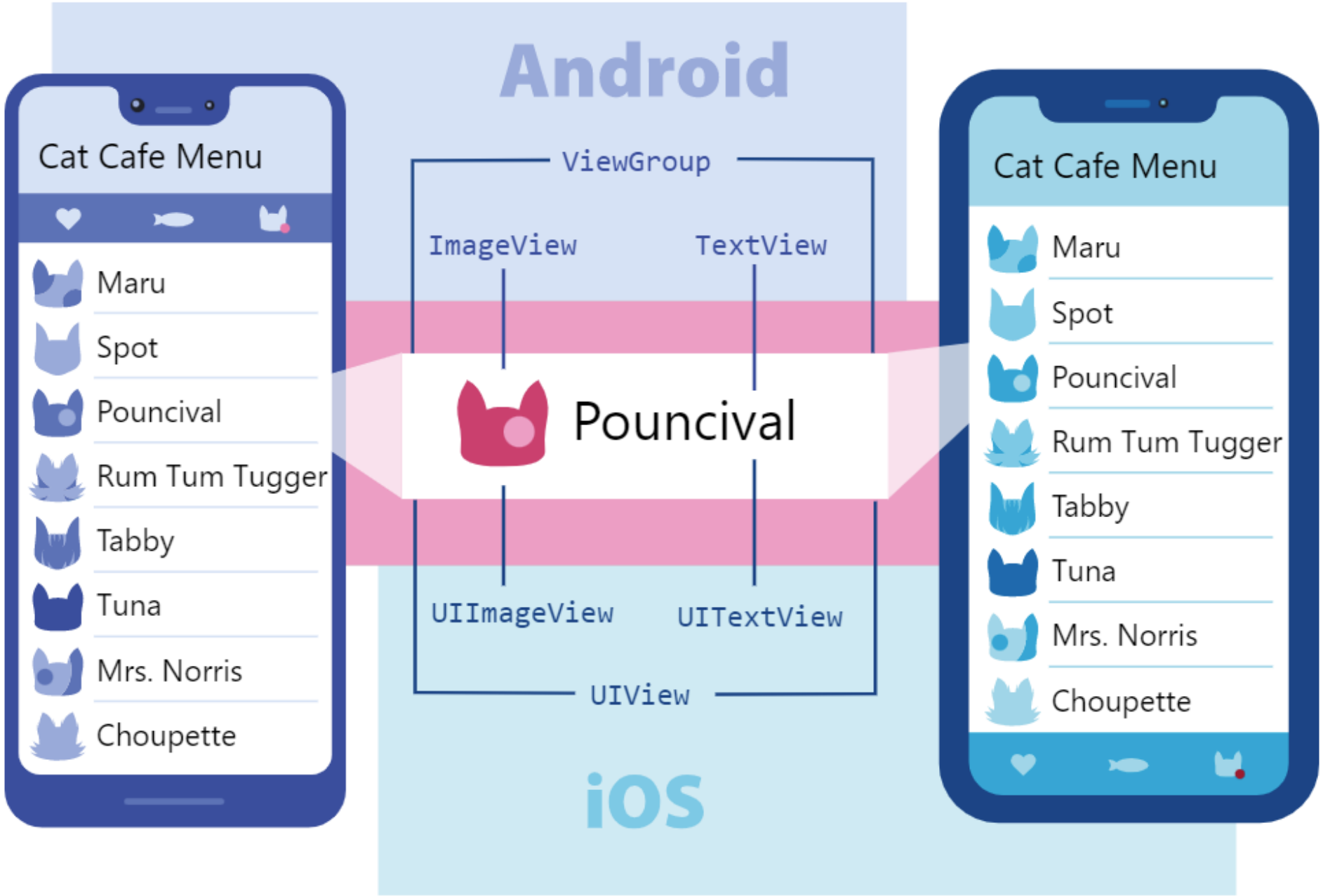

VinaPhone
20:05
20%
Học Tập
Hình Ảnh
10th ANNIVERSARY
SUMMER STARS CONTEST
SUMMER STAR 2022 - WORDLIST KID 12 (TỪ VỰNG)
THI TIẾNG ANH HỌC KỲ 2
[HK2 - KHỐI 9] ĐỀ CƯƠNG ÔN THI HỌC KỲ 2 ĐÃ CÓ TRÊN STUDENT PORTAL

Types of Components

- Core components
- Community components
- Your native components

Core Components

- Built-in, ready to use from React Native
- Is translated into native iOS and native Android components



Just a sampling of the many views used in Android and iOS apps.

Basic Components

REACT NATIVE UI COMPONENT	ANDROID VIEW	IOS VIEW	WEB ANALOG	DESCRIPTION
`<View>`	`<ViewGroup>`	`<UIView>`	A non-scrolling `<div>`	A container that supports layout with flexbox, style, some touch handling, and accessibility controls
`<Text>`	`<TextView>`	`<UITextView>`	`<p>`	Displays, styles, and nests strings of text and even handles touch events
`<Image>`	`<ImageView>`	`<UIImageView>`	`<img>`	Displays different types of images
`<ScrollView>`	`<ScrollView>`	`<UIScrollView>`	`<div>`	A generic scrolling container that can contain multiple components and views
`<TextInput>`	`<EditText>`	`<UITextField>`	`<input type="text">`	Allows the user to enter text

View Component

- Is the most common element in react native
- View as a container element
- Library

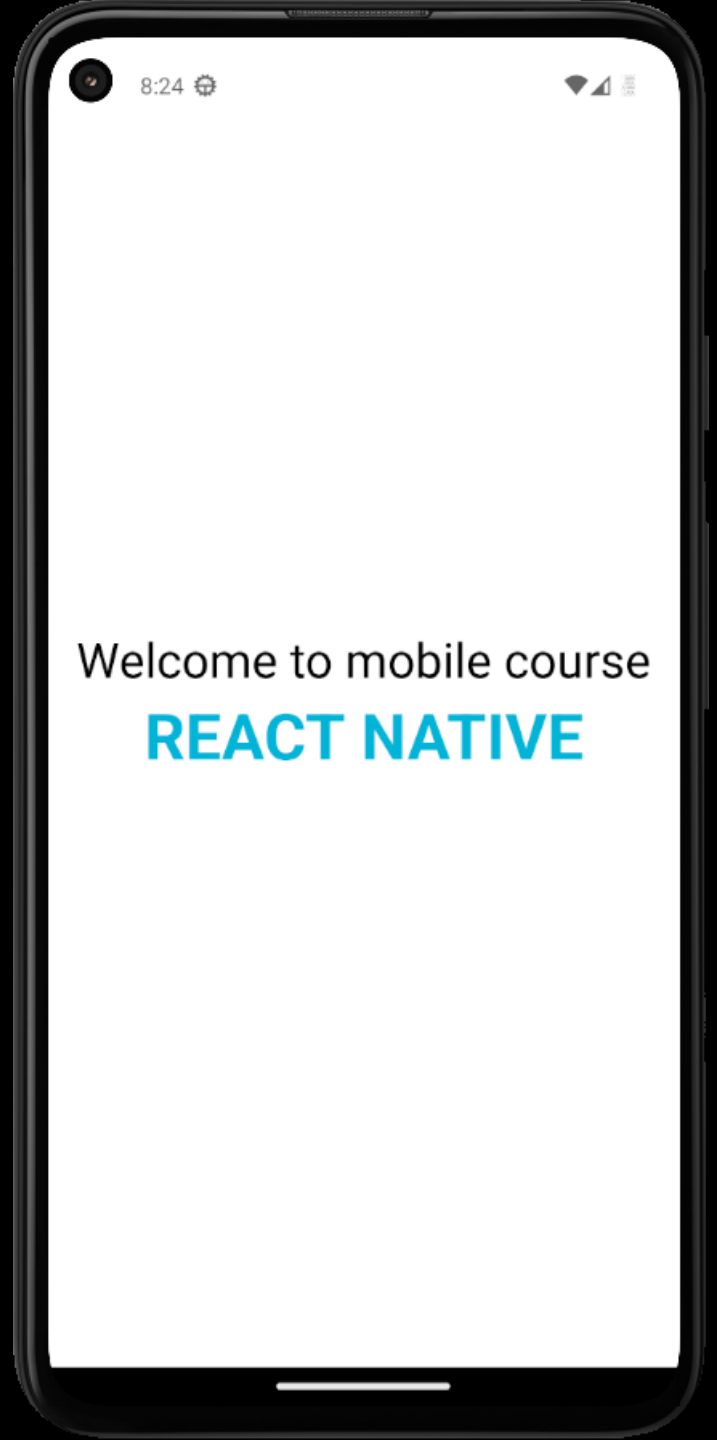
```
import {View } from 'react-native';
```

Text Component

- Displaying text
- Supports nesting, styling, and touch handling
- Event
  - OnPress
  - OnLongPress
  - Selectable
- Library

```
import {Text } from 'react-native';
```

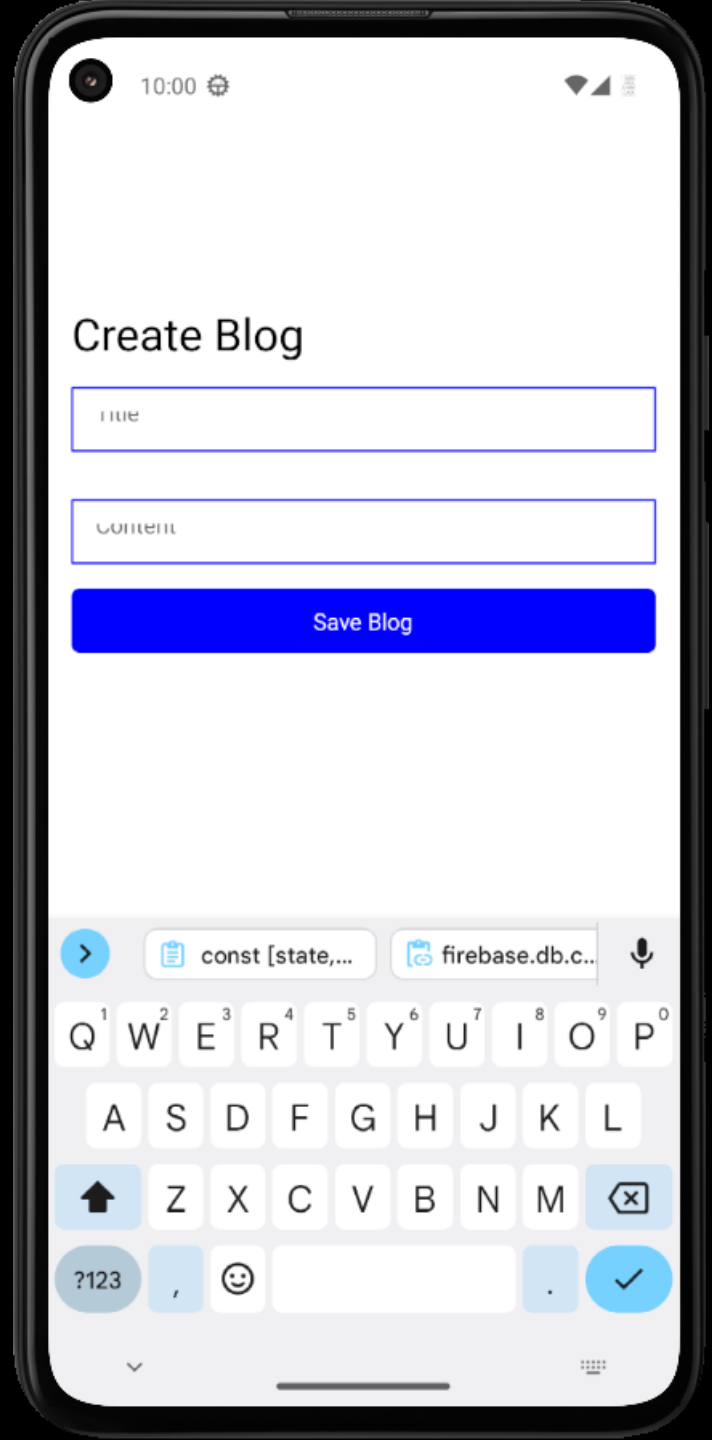
```jsx
<View style={styles.container}>
   <Text style={styles.title}>
      Welcome to mobile course</Text>
   <Text style={styles.subTitle}>
      REACT NATIVE</Text>
</View>


const styles = StyleSheet.create({
    container: {
        flex: 1,
        backgroundColor: '#fff',
        alignItems: 'center',
        justifyContent: 'center',
    },
    title: {
        fontSize: 30, color: '#000'
    },
    subTitle: {
        fontSize: 40, fontWeight: 'bold',
color: '#00b4d8'
    }
});
```



Text Input Component

- TextInput component for inputting text into the app via a keyboard
- Properties
  - value
  - placeholder
  - keyboardType
  - editable
- Events
  - onChangeText
  - onSubmitEditing
  - onFocus

```
<View style={styles.containerCreate}>
            <Text style={styles.title} >Create Blog</Text>
            <View style={styles.inputGroup}>
                <TextInput style={styles.inputText}
                    placeholder='Title'
                />
            </View>
            <View style={styles.inputGroup}>
                <TextInput style={styles.inputText}
                    placeholder='Content'
                />
            </View>
            <TouchableOpacity style={styles.btn}>
                <Text style={styles.btnTitle}>Save Blog</Text>
            </TouchableOpacity>
</View>
```



```
const styles = StyleSheet.create({

    containerCreate: {
        flex: 1,
        justifyContent: 'center',
        backgroundColor: '#FFF',
        padding: 15
    },
    title: {
        fontSize: 28,
        fontWeight:'bold'
    },
    img: {
        width: 100,
        height: 100,

    },
    inputGroup: {
        marginVertical: 15
    },
    inputText: {
        height: 40,
        borderWidth: 1,
        borderColor: '#0000FF',
        padding: 15
    },
    btn: {
        backgroundColor: '#0000FF',
        alignItems: 'center',
        justifyContent: 'center',
        height: 40,
        borderRadius: 5
    },
    btnTitle: {
        color: '#FFF'
    }
})
```

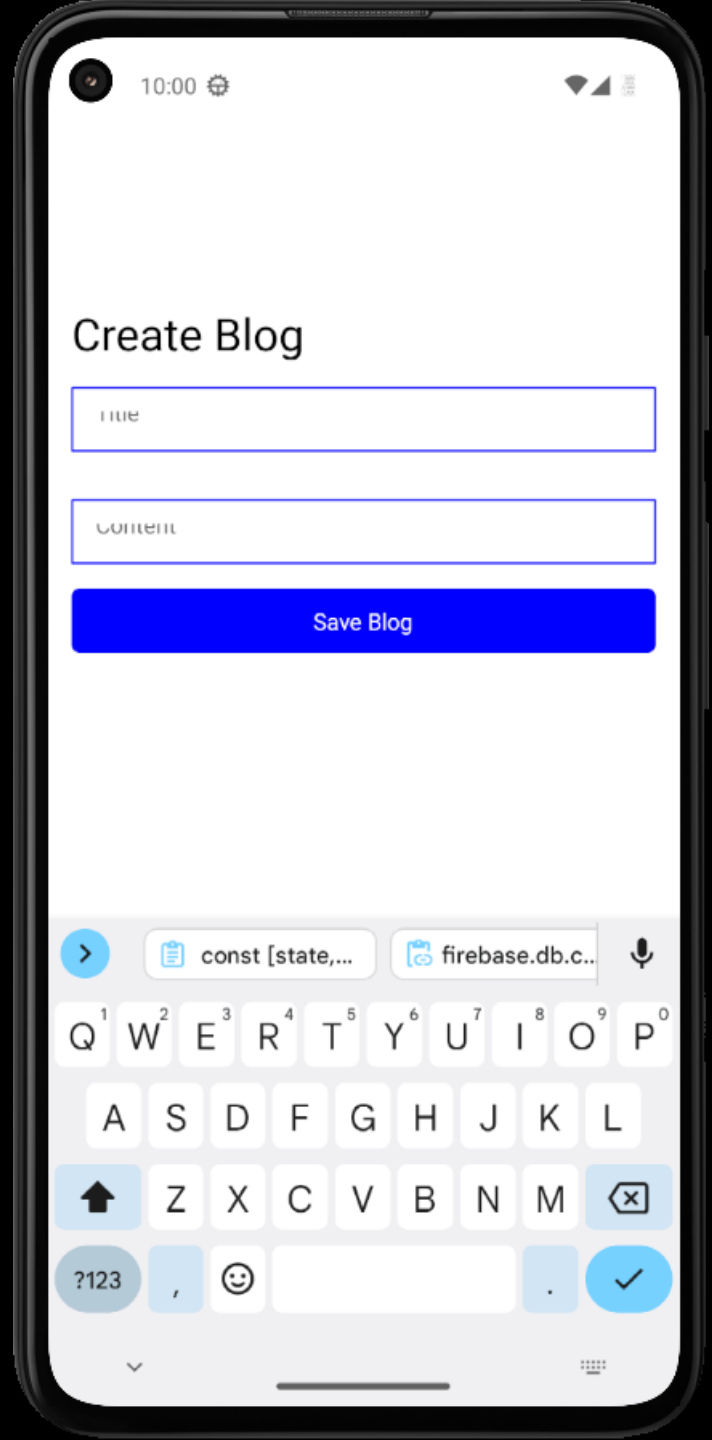


Image Component

- Render image
- Local image

```
<Image source = {require('./app/assets/logo.png')} />
```

- Network image

```
<Image
      source =
{{uri:'https://pbs.twimg.com/profile_images/486929358120964097/gNLINY67
_400x400.png'}}
      style = {{ width: 200, height: 200 }} />
```

- Library

```
import {Image } from 'react-native';
```

```
const VusView = ({ }) =>
{
    return (
        <View style={styles.containerVusView}>
            <Image source={require("../assets/vuslogo.png")}
                style={styles.img} />
        </View>
    );
};

const styles = StyleSheet.create({
    containerVusView: {
        flex: 1,
        justifyContent: 'center',
        alignItems: 'center',
        backgroundColor: 'rgb(185,19,26)',
    },
    title: {
        fontSize: 28,
        fontWeight: 'bold'
    },
    img: {
        width: 100,
        height: 100,
    }
})
```

ScrollView Component

- Generic scrolling container
- Contain multiple components and views
- Scrollable items can be heterogeneous
- Library

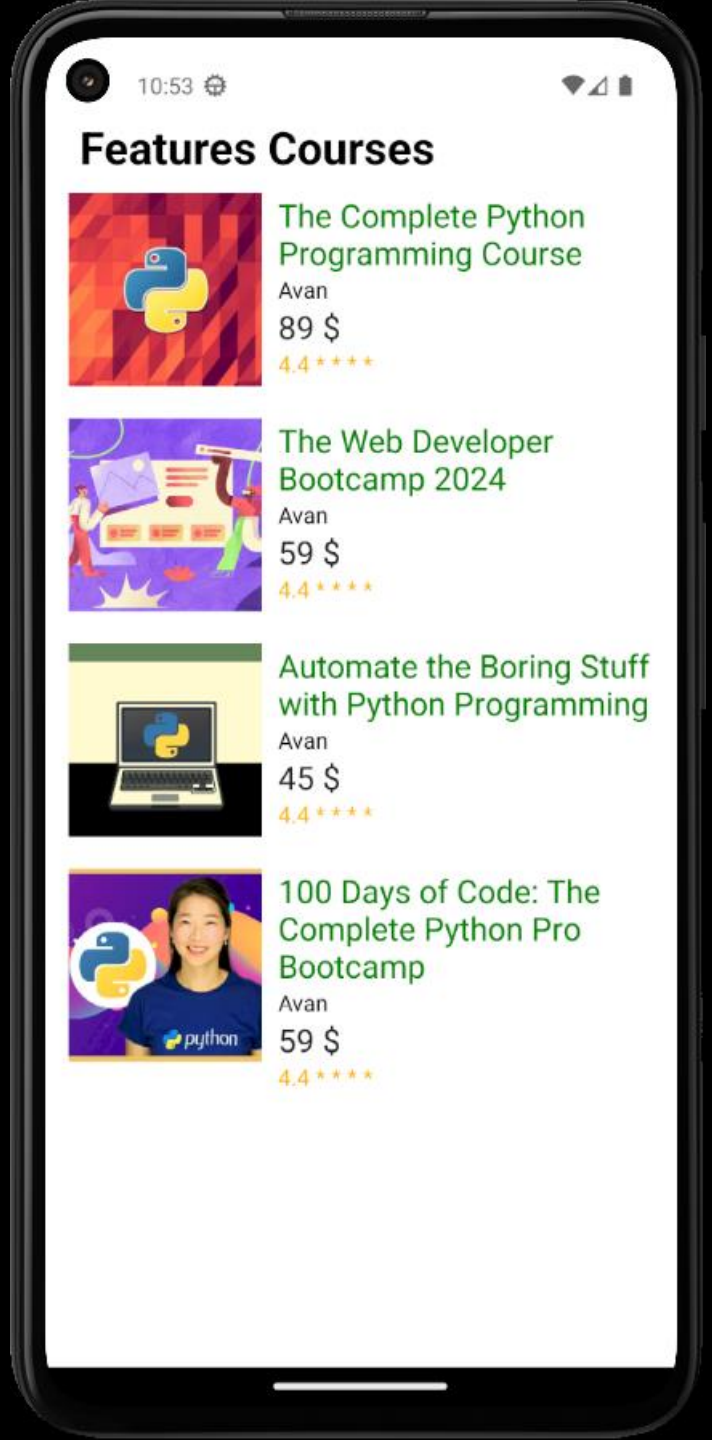
```
import {ScrollView } from 'react-native';
```

FlatList Component

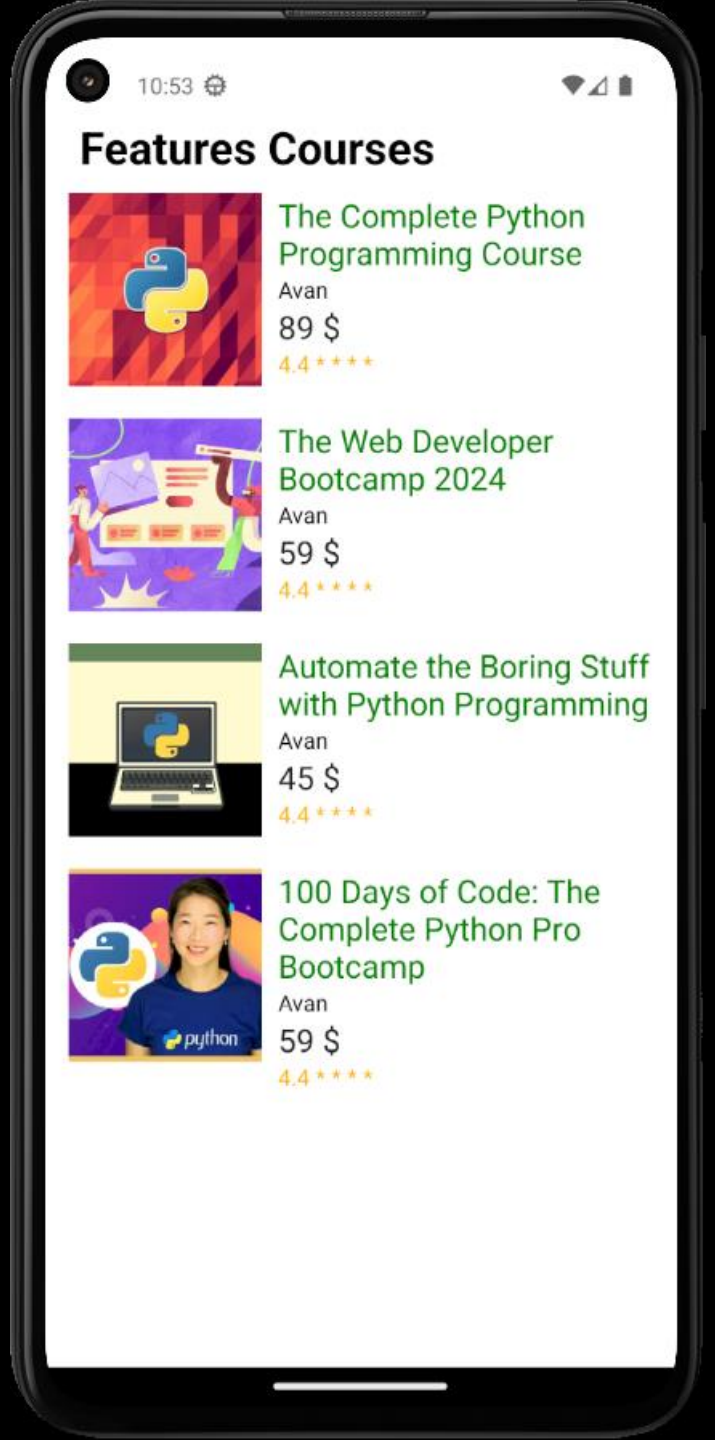
- Render multiple items
- Properties
  - renderItem
  - keyExtractor
  - ItemSeparatorComponent
  - ListHeaderComponent
  - data
- Library

```
import { FlatList } from "react-native";
```

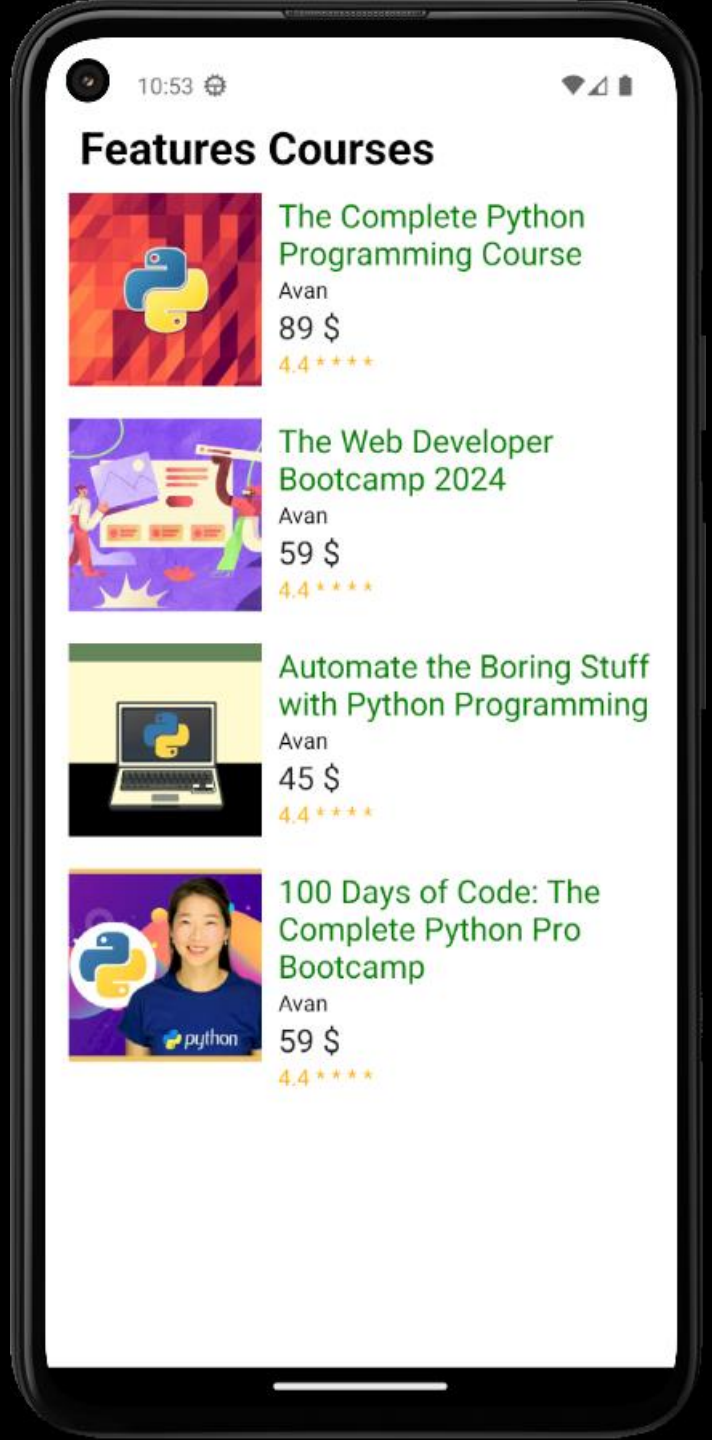
```
<View style={styles.listItemContainer}>
          <FlatList
              data={items}
              ListHeaderComponent={HeaderComponent}
              renderItem={ItemComponent}>
          </FlatList>
</View>
```



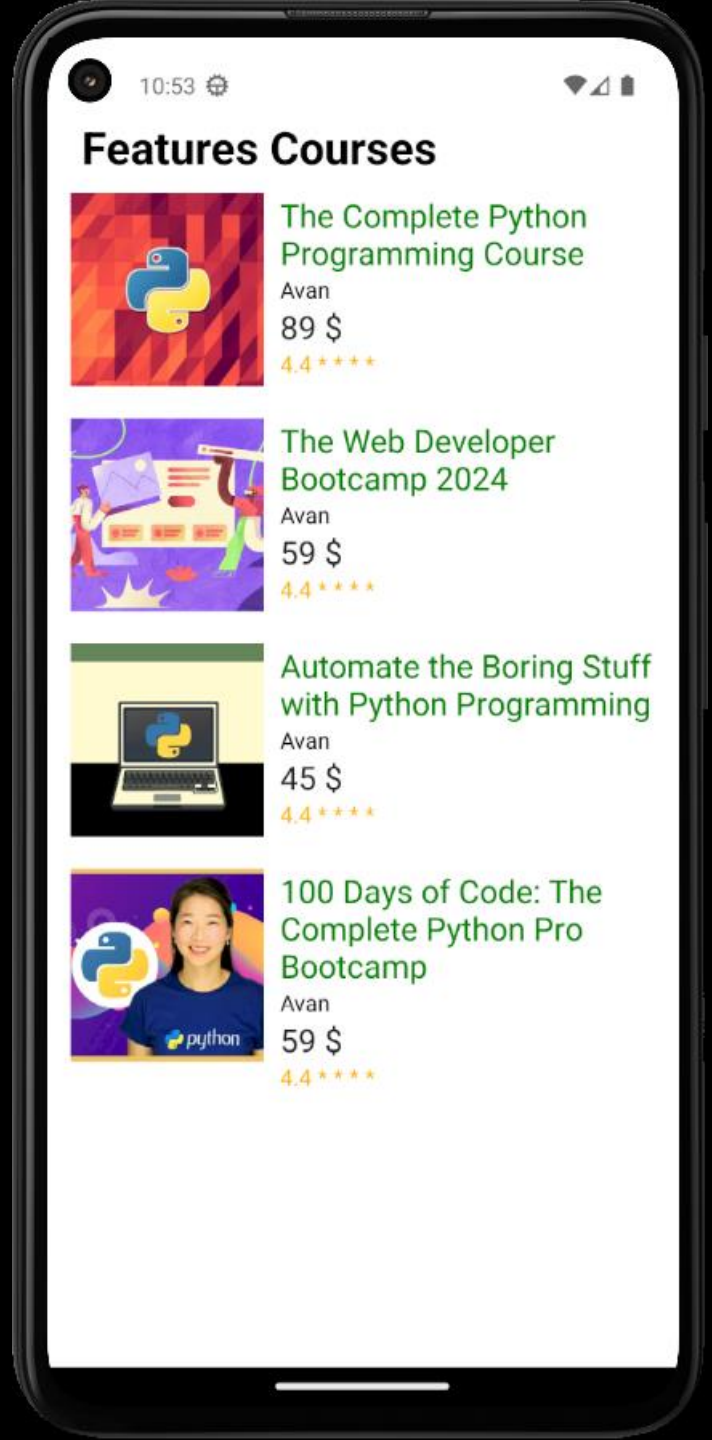
```
const items = [
    {
        id: 1,
        name: "The Complete Python Programming Course",
        img: "https://img-
c.udemycdn.com/course/480x270/394676_ce3d_5.jpg",
        teacher: 'Avan',
        price: 89,
        rating: 4.4
    },
    {
        id: 2,
        name: "The Web Developer Bootcamp 2024",
        img: "https://img-
c.udemycdn.com/course/750x422/625204_436a_3.jpg",
        teacher: 'Avan',
        price: 59,
        rating: 4.4
    },
    // more
];
```



```jsx
const HeaderComponent = () =>
{
    return (
        <View>
            <Text style={styles.titleCourses}>
                Features Courses </Text>
        </View>
    )
}
```



```
const ItemComponent = ({ item }) =>
{
    return (
        <View style={styles.itemContainer}>
            <View style={styles.imgCourseContainer}>
                <Image source={{ uri: item.img }}
                    style={styles.imgCourse} />
            </View>
            <View style={styles.courseDetailContainer}>
                <Text style={styles.itemName}>
                    {item.name}</Text>
                <Text>{item.teacher}</Text>
                <Text style={styles.itemPrice}>
                    {item.price} $</Text>
                <Text style={styles.itemRating}>
                    {item.rating} * * * *</Text>
            </View>

        </View>
    )
}
```



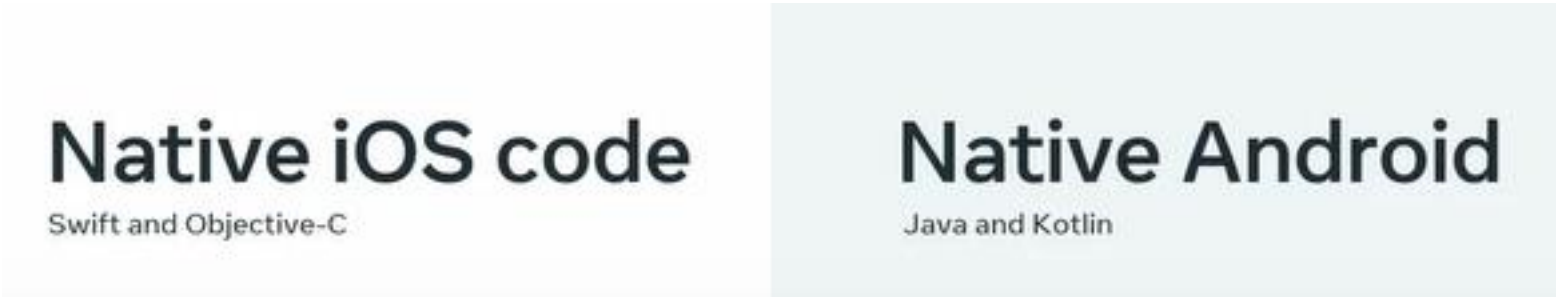
Community Components

- React navigation
- React Native screens
- React Native maps
- React Native video, and many more
- Native Directory

https://reactnative.directory/

Your Native Components

- Are written in native code



2

STYLING COMPONENTS USING STYLESHEET

REACT NATIVE STYLES

- An abstraction similar to CSS StyleSheets
- Import { **StyleSheet**} from 'react-native'
- Use **StyleSheet.create()** to define several styles
- Naming convention: names are written using camel casing, e.g. backgroundColor
- Using **style** property to declare your style

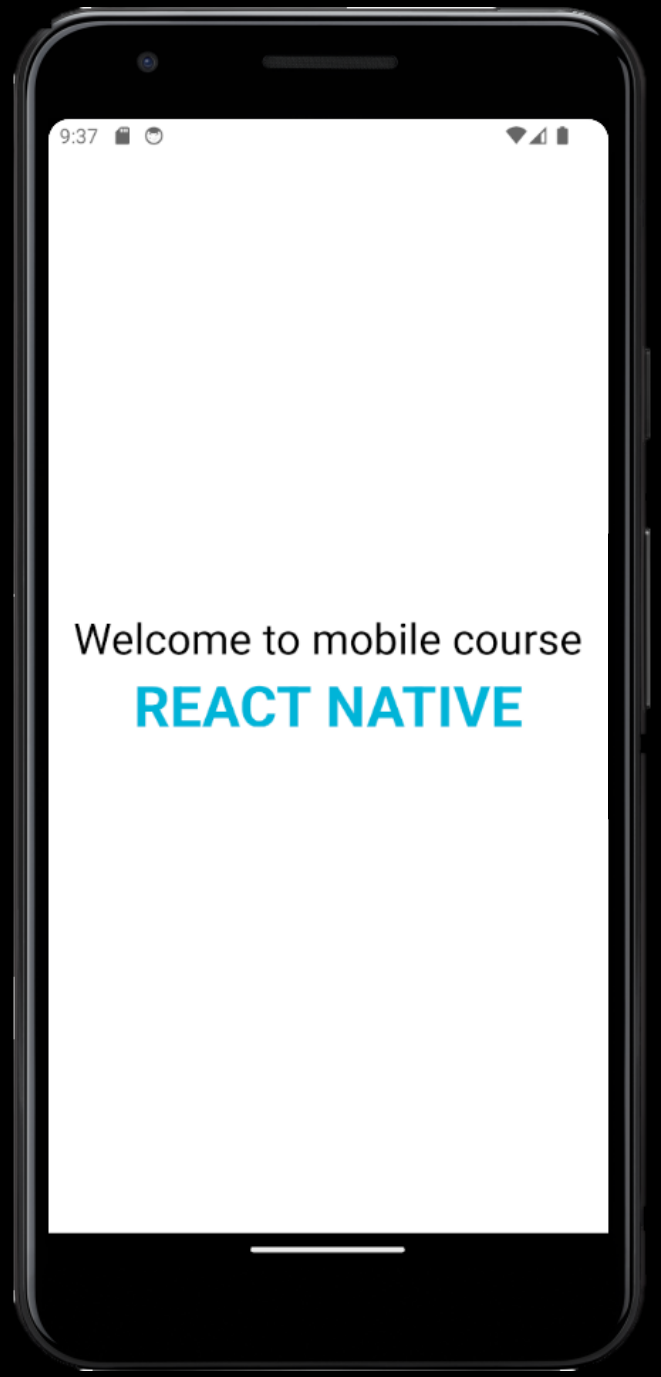
```
<View style={{
      flex: 1,
      backgroundColor: '#fff',
      alignItems: 'center',
      justifyContent: 'center'
    }}>
      <Text style={{
                  fontSize: 30,
                  color: '#424242’
            }}>
            Welcome to my course
      </Text>
      <Text style={{
                  fontSize: 40,
                  color: '#00b4d8’
            }}>
            React natives
      </Text>
</View>
```

```
<View style={{
      flex: 1,
      backgroundColor: '#fff',
      alignItems: 'center',
      justifyContent: 'center'
    }}>
      <Text style={{
                  fontSize: 30,
                  color: '#424242’
            }}>
            Welcome to my course
      </Text>
      <Text style={{
                  fontSize: 40,
                  color: '#00b4d8’
            }}>
            React natives
      </Text>
</View>
```

```
const styles = StyleSheet.create({
  container: {
    flex: 1,
    backgroundColor: '#fff',
    alignItems: 'center',
    justifyContent: 'center',
  },
  txtTitle: {
```

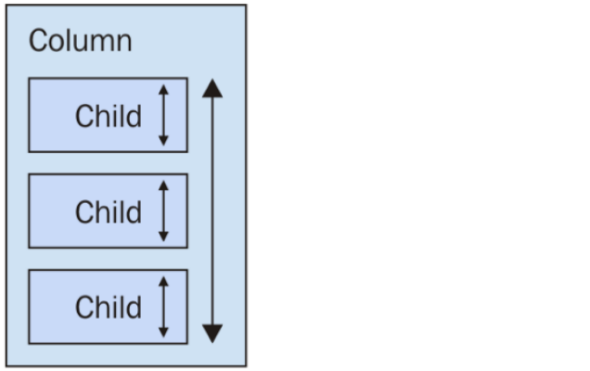
```
const styles = StyleSheet.create({
  container: {
    flex: 1,
    backgroundColor: '#fff',
    alignItems: 'center',
    justifyContent: 'center',
  },
  txtTitle: {
    fontSize: 30,
    color: '#424242'
  },
  txtSubTitle: {
    fontSize: 40,
    color: '#00b4d8'
  }

});
```

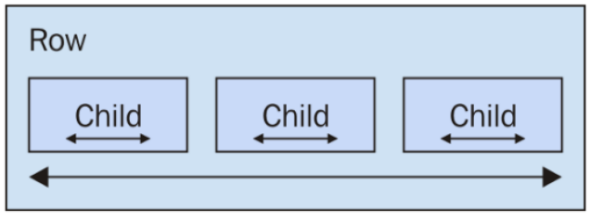
```
<View style={styles.container}>
            <Text style={styles.txtTitle}>
              Welcome to my course
            </Text>
            <Text style={styles.txtSubTitle}>
              React natives
            </Text>
</View>
```



LAYOUTS WITH FLEXBOX

- Flexbox containers have a direction, either Column (up/down) or Row (left/right)



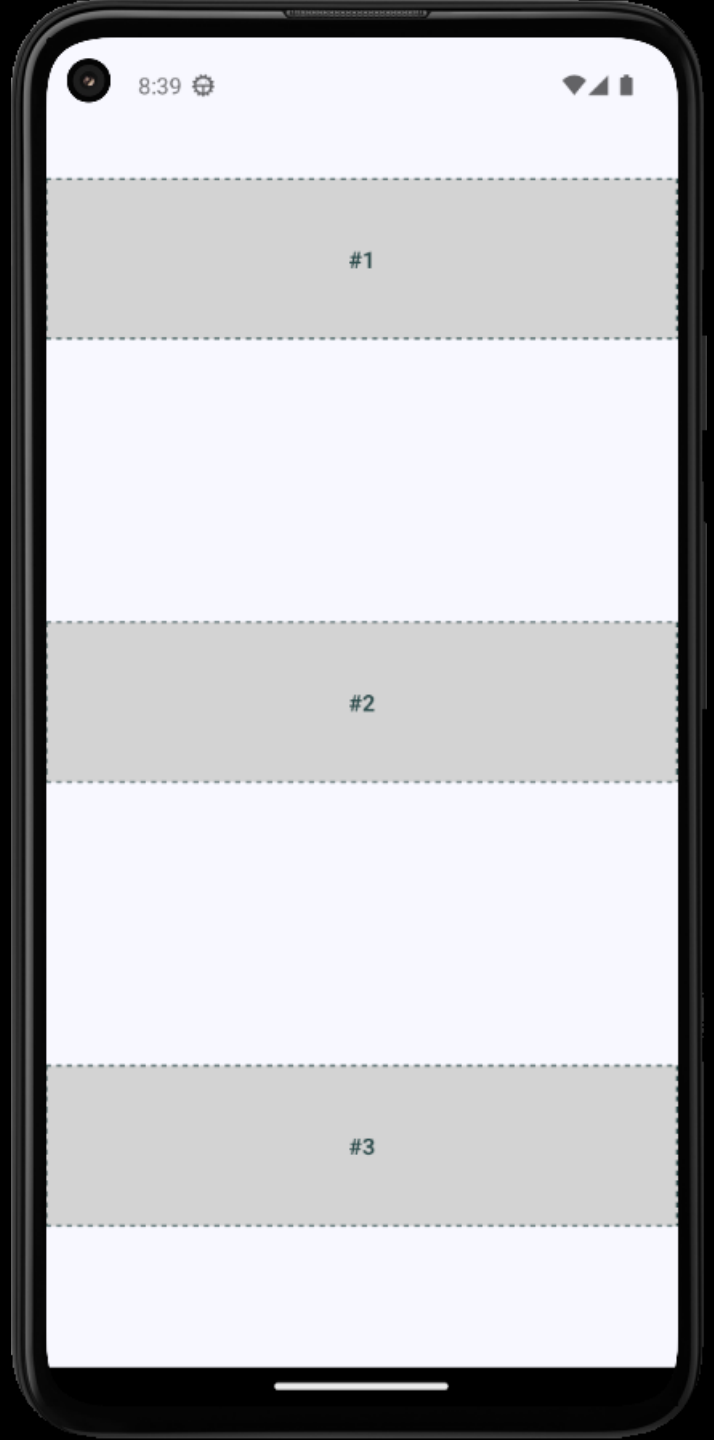


Property	Values	Description
flexDirection	'column', 'row'	Used to align its elements vertically or horizontally.
justifyContent	'center', 'flex-start', 'flex-end’, 'space-around’, 'spacebetween'	Used to distribute the elements inside the container.
alignltems	'center', 'flex-start', 'flex-end’, 'stretched'	Used to distribute the element inside the container along the secondary axis (opposite to flexDirection).

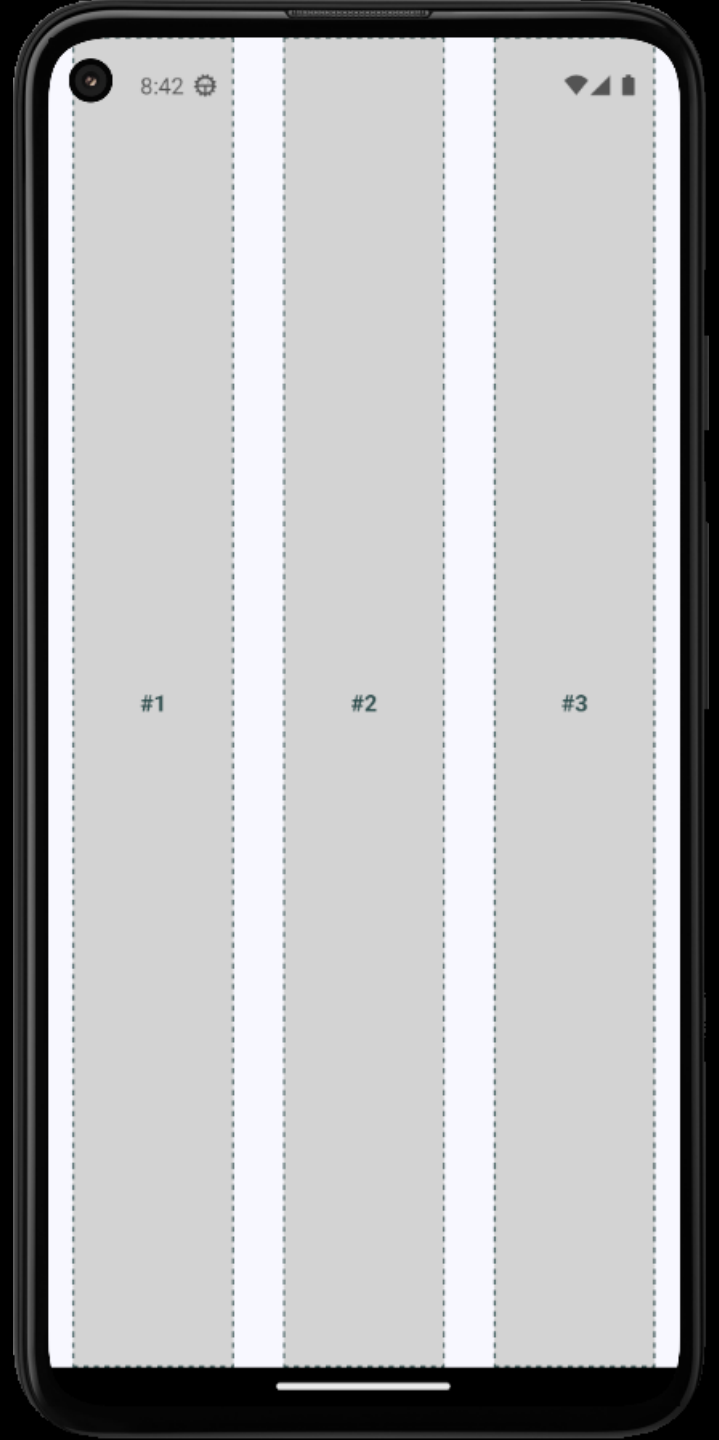
EXAMPLE

```
const FlexLayoutDemo = () =>
{
    return (

        <View style={styles.container}>
            <View style={styles.box}>
                <Text style={styles.boxText}>#1</Text>
            </View>
            <View style={styles.box}>
                <Text style={styles.boxText}>#2</Text>
            </View>
            <View style={styles.box}>
                <Text style={styles.boxText}>#3</Text>
            </View>
        </View>
    );
}
```

```
const styles = StyleSheet.create({
    container: {
        flex: 1,
        flexDirection: “column",
        alignItems: "center",
        justifyContent: "space-around",
        backgroundColor: "ghostwhite",
    },
    box: {
        height: 100,
        backgroundColor: "lightgray",
        borderWidth: 1,
        borderStyle: "dashed",
        borderColor: "darkslategray",
        alignSelf: "stretch",
        justifyContent: "center",
        alignItems: "center",
    },
    boxText: {
        color: "darkslategray",
        fontWeight: "bold"
    }
});
```



```
const styles = StyleSheet.create({
    container: {
        flex: 1,
        flexDirection: “row",
        alignItems: "center",
        justifyContent: "space-around",
        backgroundColor: "ghostwhite",
    },
    box: {
        width: 100,
        backgroundColor: "lightgray",
        borderWidth: 1,
        borderStyle: "dashed",
        borderColor: "darkslategray",
        alignSelf: "stretch",
        justifyContent: "center",
        alignItems: "center",
    },
    boxText: {
        color: "darkslategray",
        fontWeight: "bold"
    }
});
```



3

REACT NATIVE HOOK

Function component
Class component

Function Component

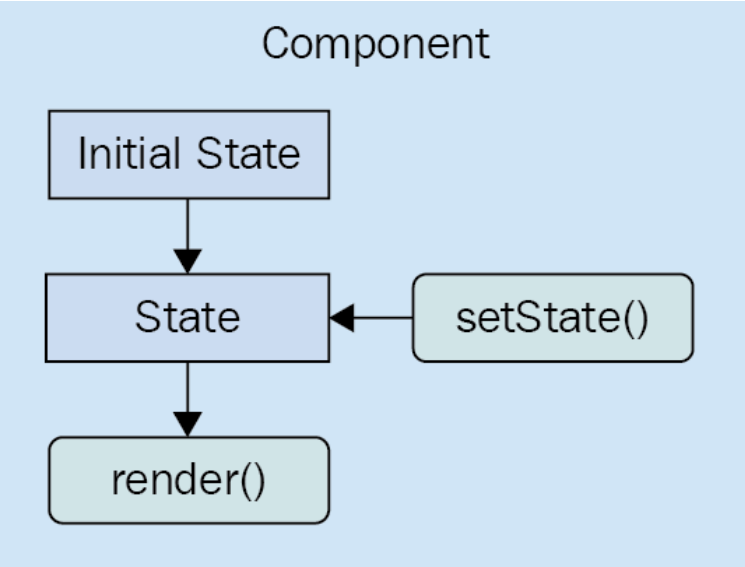
```
import React from 'react';
import { Text, View } from 'react-native';

const HelloWorldApp = () => {
  return (
    <View style={{
      flex: 1,
      justifyContent: 'center',
      alignItems: 'center'
    }}>
      <Text>Hello, world!</Text>
    </View>
  );
}

export default HelloWorldApp;
```

Class Component

```
import React, { Component } from 'react';
import { Text, View } from 'react-native';

class HelloWorldApp extends Component {
  render() {
    return (
      <View style={{
          flex: 1,
          justifyContent: "center",
          alignItems: "center"
        }}>
        <Text>Hello, world!</Text>
      </View>
    );
  }
}

export default HelloWorldApp;
```

WHAT IS COMPONENT STATE ?

- Is the dynamic part of a React component
- Declare the initial state of a component, which changes over time



PROPS vs STATE

- Data inside Reactnative Components is managed by state and props
- **State** is **mutable** while **props** are **immutable**
- **State** can be updated in the future while **props** cannot be updated

PROPS

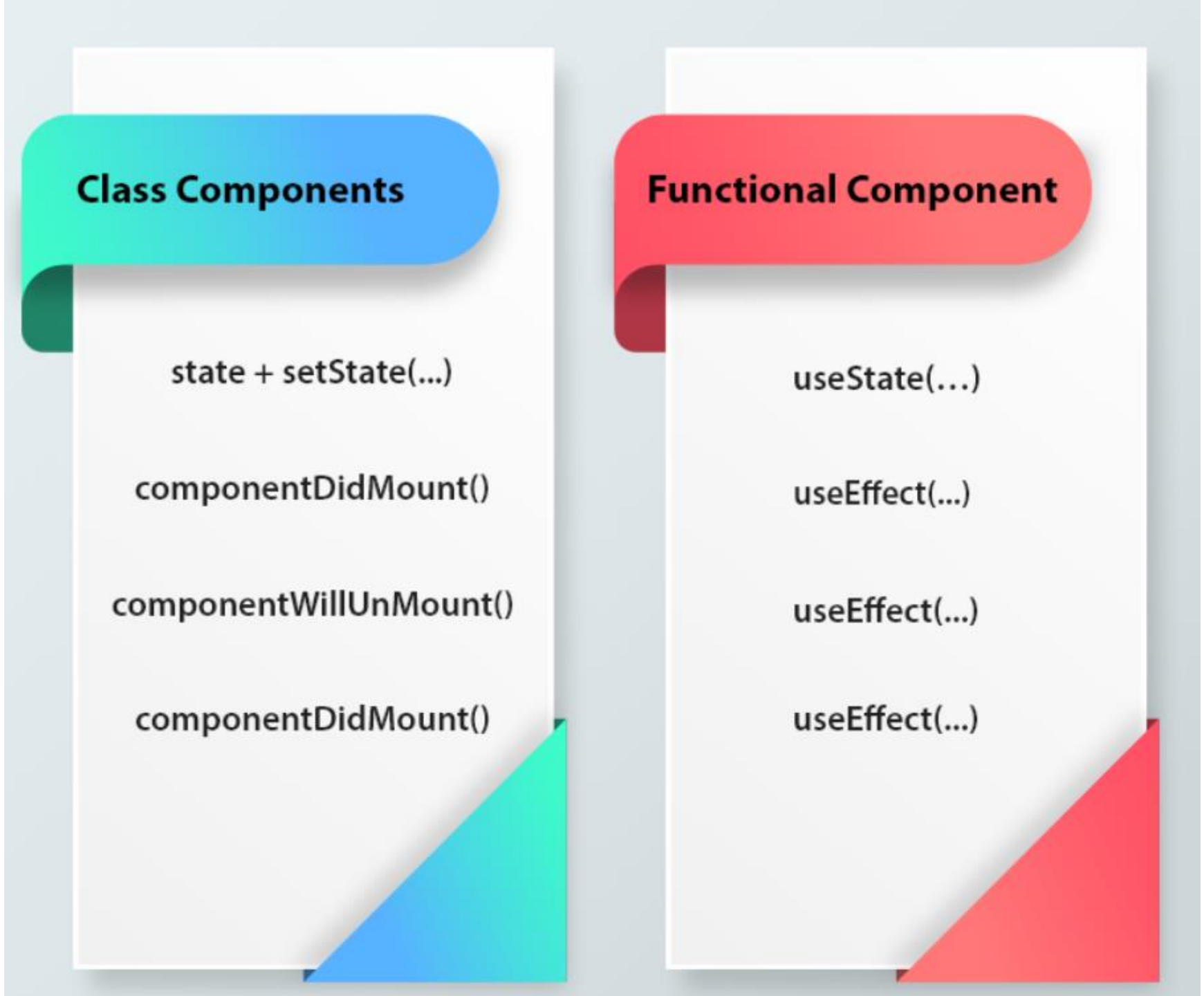
- Passed as an object to a component and used to compute the returned node
- Changes in these props will cause a recomputation of the returned node (“render”)

STATE

- Adds internally-managed configuration for a component
  - '**this.state**' is a class property on the component instance
  - Can only be updated by invoking '**this.setState()**'
- **setState()** calls are batched and run asynchronously
- Pass an object to be merged, or a function of previous state changes in state also cause re-renders

REACT NATIVE HOOK

- A new feature addition in React Native version 16.8
- Are functions to use state and lifecycle methods inside functional components
- No more complex lifecycle methods
- Simpler code, avoid the whole confusion with '**this**' keyword
- Basic Hooks
  - useState
  - useEffect
  - useRef


Class Components
state + setState(...)
componentDidMount()
componentWillUnMount()
componentDidMount()
Functional Component
useState(...)
useEffect(...)
useEffect(...)
useEffect(...)

```
  const CountView= ({})=> {
  const [count, setCount] = useState(0);
  return (
   <View style={styles.containerCountView}>
      <Text style={{fontSize: 28, fontWeight:'bold'}}>
     CountView </Text>
      <Text style={{paddingBottom:10}}>
            You clicked {count} times </Text>
      <Button title="Click me"
            onPress={() => setCount(count + 1)}/>
    </View>

  );
}
  const styles = StyleSheet.create({
     containerCountView: {
        flex:1,
         justifyContent: 'center',
         alignItems:'center',
         height: 200,
         backgroundColor: '#fff',
         marginBottom:10
     }
  })
```